

TradePilot v6 Live Trading Threat Analysis

- Why This Report Matters
- CATEGORY 1: Kite API Security
- CATEGORY 2: Financial Threats (Real Money)
- CATEGORY 3: Network Security
- CATEGORY 4: Infrastructure Threats
- CATEGORY 5: Quantum Computing Threats
- CATEGORY 6: Data Protection
- CATEGORY 7: Operational Security
- CATEGORY 8: Social Engineering and Phishing
- CATEGORY 9: Supply Chain Attacks
- CATEGORY 10: Regulatory and Legal
- Quantum-Grade Security Architecture
- Summary: Threat Severity Matrix
- Final Recommendation

TradePilot v6 Live Trading Threat Analysis

Comprehensive Security Assessment for the Paper-to-Live Trading Transition

Project	TradePilot v6 – The Machine
Version	v1.0.0
Status	Planning Phase – Pre-Live
Created	2026-04-14
Updated	2026-04-14
Classification	CONFIDENTIAL – Founder Eyes Only
Prepared For	Soumya Swain, Co-Founder
Contact	support@devpilot.co.in

Why This Report Matters

TradePilot is about to cross the most dangerous line in fintech: **from pretend money to real money.**

In paper trading, a bug means a wrong number on a screen. In live trading, a bug means real rupees vanishing from your Zerodha account in milliseconds. There is no “undo” button on the stock market.

This report covers every threat that could cost you money, get you in legal trouble, or compromise the system – explained in plain language with real-world analogies. Every threat includes what it is, the worst-case scenario specific to TradePilot, how likely it is, and exactly what to do about it.

The report also covers quantum computing threats – attacks that do not exist today but could break current security in 5-10 years. Building quantum resistance now is like installing earthquake-proof foundations in a building – it costs a little more upfront but prevents catastrophe later.

CATEGORY 1: Kite API Security

The Zerodha Kite API is the bridge between TradePilot’s brain and the stock market. Every buy order, sell order, and position check flows through this bridge. If the bridge is compromised, an attacker controls your trading account.

1.1 How Kite API Authentication Works

Simple analogy: Think of Kite API access like getting into a high-security office building every morning.

The authentication has three layers:

Layer	What It Is	Analogy	Lifespan
API Key	A unique identifier for your app	Your employee ID badge number – it identifies you but does not open doors	Permanent (until you regenerate it)
API Secret	A password paired with your API key	The master key to the building – if someone copies this, they can make new access passes	Permanent (until you regenerate it)
Request Token	A one-time code from Zerodha login	The code the security desk gives you after checking your ID	Single use – expires in minutes
Access Token	The daily session pass	Your daily entry pass – works all day, expires at midnight	Valid until ~6:00 AM next day

The daily dance works like this:

1. Every morning before market opens (9:15 AM), your app opens a Zerodha login page

2. You (or the system) logs in with Zerodha username + password + TOTP (time-based one-time password from Google Authenticator)
3. Zerodha gives back a “request token” – a one-time code
4. Your app combines the request token + API key + API secret, hashes them together, and sends them to Zerodha
5. Zerodha validates everything and returns an “access token”
6. This access token is used for ALL API calls that day – every order, every position check, every data fetch
7. At ~6:00 AM next morning, the token expires and you repeat the process

Why this matters for TradePilot: The v6 system needs to automate this daily login. That means storing the API secret somewhere the system can access it. This storage is the first point of attack.

1.2 What Happens If Someone Steals the Kite Access Token

Threat: An attacker obtains your daily access token.

Analogy: Someone steals your daily office pass. They can enter any room you have access to – until the pass expires at midnight.

What they CAN do with a stolen access token:

	Action	Damage
☐	Place buy orders	Buy stocks you did not want – you are stuck paying for them
☐	Place sell orders	Sell stocks you are holding – at whatever price the market gives
☐	Modify existing orders	Change your stop-loss to a terrible price, then let the market hit it
☐	Cancel pending orders	Remove your protective stop-losses, leaving positions exposed
☐	View your portfolio	See every stock you hold, every trade you made, your exact P&L
☐	View your funds	See your account balance, margin available, and collateral
☐	Place F&O orders	Enter options and futures contracts with unlimited loss potential

What they CANNOT do (Zerodha protections):

Protection
☐

Protection	
	Withdraw money to a bank account (requires separate Zerodha login + 2FA on their website)
☐	Transfer shares to another demat account (requires CDSL/NSDL authentication)
☐	Change your Zerodha password or linked bank account
☐	Access your Aadhaar, PAN, or other KYC documents

Worst case for TradePilot: An attacker places 50 buy orders for illiquid penny stocks totaling Rs 20-30 lakh. By the time you notice (minutes to hours), the orders have executed. You are now holding worthless stocks. The attacker was the one selling those penny stocks to you – a classic “pump and dump” using your money.

Likelihood: MEDIUM (requires access to the server where the token is stored, or intercepting the token in transit)

The solution:

1. **Encrypt the access token at rest** – store it in an encrypted vault (HashiCorp Vault or AWS Secrets Manager), never in a plain text file or environment variable
2. **Token rotation monitoring** – log every time the access token is used, alert if it is used from an unexpected IP address
3. **IP whitelisting** – Zerodha allows you to restrict API access to specific IP addresses. Set this to only your server’s IP
4. **Minimal permissions** – when registering the Kite app, request only the permissions you need (no fund transfer permissions)
5. **Token expiry awareness** – build monitoring that confirms the token expires each night and a fresh one is generated each morning

1.3 API Key vs Secret vs Access Token – Danger Ranking

Credential	If Stolen, Damage Level	Analogy
API Key	LOW – alone it does nothing	Your employee badge number. Useless without the badge itself
API Secret	CRITICAL – can generate new access tokens	The master mold for making badges. Someone can create unlimited valid badges
Access Token	HIGH – full account control for the day	Today’s badge. Full

Credential	If Stolen, Damage Level	Analogy
API Key + Secret	CRITICAL – permanent access until you regenerate them	access until midnight The badge AND the mold. They never need to steal your daily pass again

Worst case for TradePilot: If the API secret is stolen, the attacker can generate fresh access tokens every day forever (until you notice and regenerate the secret). This is far worse than stealing a single day's access token.

Likelihood: MEDIUM-HIGH (the secret must be stored somewhere the application can read it – that storage is the target)

The solution:

1. **Never store the API secret in code, config files, or environment variables** – use a hardware security module (HSM) or cloud secrets manager
2. **Rotate the API secret quarterly** – even if you do not suspect compromise
3. **Audit access** – log every time the secret is read from the vault
4. **Emergency procedure** – document exactly how to regenerate the API key and secret in under 5 minutes (Zerodha developer portal > My Apps > Regenerate)

1.4 Session Hijacking Risks

Threat: An attacker intercepts the communication between TradePilot and Zerodha and “takes over” the session.

Analogy: Imagine you are on a phone call with your stockbroker. Someone taps into the phone line, disconnects you silently, and continues the conversation pretending to be you. The broker has no idea the voice changed.

How it could happen:

Vector	How	Likelihood
Network sniffing on shared WiFi	Attacker on same coffee shop WiFi intercepts HTTP traffic	LOW (Kite API uses HTTPS)
Compromised server network	If your cloud server's network is breached, internal traffic could be intercepted	MEDIUM
TLS downgrade attack		LOW (Zerodha enforces TLS 1.2+)

Vector	How	Likelihood
Local process memory dump	Force the connection to use an older, broken encryption standard Malware on the server reads the access token from the running program's memory	MEDIUM

Worst case for TradePilot: The attacker hijacks the session mid-trading-day, places rogue orders, and your system does not notice because it thinks it still has a valid session. The system's own monitoring reports everything as "normal" because the session is still active.

Likelihood: LOW-MEDIUM

The solution:

1. **Certificate pinning** – hardcode the Zerodha API server's SSL certificate fingerprint in your code. If the certificate changes (man-in-the-middle), refuse to connect (see Category 3 for details)
2. **Session heartbeat** – every 60 seconds, make a lightweight API call (GET /user/profile) and verify the response. If the session is hijacked, the attacker's actions will change the account state, and your heartbeat will detect inconsistencies
3. **Process isolation** – run the trading engine in a container with no other processes. This reduces the attack surface for memory dumps
4. **Memory encryption** – use encrypted memory pages for storing the access token in RAM (available on modern Linux kernels)

1.5 Rate Limit Abuse

Threat: TradePilot's algorithm makes too many API calls, hitting Zerodha's rate limits, and orders fail at critical moments.

Analogy: You are at a bank that allows 10 transactions per minute. Your automated system tries to make 50 transactions per minute. The bank blocks you – including the critical "sell everything" transaction you needed when the market crashed.

Zerodha Kite API rate limits:

Endpoint Type	Limit	What Happens If Exceeded
Order placement	10 orders/second	HTTP 429 error – order is rejected
Historical data	3 requests/second	HTTP 429 – data fetch fails

Endpoint Type	Limit	What Happens If Exceeded
Quote data	1 request/ second per instrument	HTTP 429 – stale prices used
WebSocket ticks	3,000 instruments max	Connection dropped

Worst case for TradePilot: During a flash crash, the v6 system detects danger and tries to exit all 20 positions simultaneously. It fires 20 sell orders at once, exceeding the 10/second limit. Half the orders fail. Those 10 stocks continue falling. By the time the system retries (after the mandatory cooldown), prices have dropped another 2-3%. On Rs 50 lakh deployed capital, that is Rs 1-1.5 lakh in avoidable losses.

Likelihood: HIGH (this WILL happen if not designed for)

The solution:

1. **Order queue with rate limiter** – build a queue that processes maximum 8 orders/second (leaving 2/second headroom for manual interventions)
2. **Priority ordering** – during emergencies, exit largest positions first. A stop-loss on a Rs 5 lakh position is more important than one on Rs 50K
3. **Batch order API** – Zerodha's basket orders allow placing multiple orders as a single API call. Use this for bulk exits
4. **Pre-computed exit orders** – place stop-loss orders at the time of entry, not when the crash happens. Then you do not need to fire sell orders during a crisis
5. **Local rate tracking** – maintain a counter of API calls made per second. If approaching the limit, queue remaining orders with a 100ms delay between them

CATEGORY 2: Financial Threats (Real Money)

This is the category that keeps founders up at night. These are not hackers – these are bugs, design flaws, and edge cases in your own code that can drain real money.

2.1 Rogue Algorithm Placing Wrong Orders

Threat: A bug in the trading algorithm causes it to place orders that are financially destructive.

Analogy: You hire a stockbroker and give them instructions: “Buy Reliance if it goes above Rs 2,500.” But due to a typo in the instructions, they read it as “Buy Reliance 2,500 shares” (quantity instead of price). They buy 2,500 shares at Rs 2,500 each – Rs 62.5 lakh worth.

Real examples from Wall Street:

Incident	What Happened	Loss
Knight Capital (2012)		

Incident	What Happened	Loss
	A code deployment bug caused the system to buy stocks it should have sold. 45 minutes of rogue trading	\$440 million (Rs 3,600 crore)
Infosys fat finger (2023)	Accidental order on Nifty options due to wrong lot size	Rs 250 crore market impact
Samsung Securities (2018)	System issued 1,000 shares per employee instead of 1,000 won (currency) per employee	\$105 billion in phantom shares

How this could happen in TradePilot:

- A regression in `signal_engine.py` flips BUY and SELL signals
- The `position_sizer.py` calculates quantity based on old prices cached in `prototype/data/cache/`
- A change in `composite_scorer.py` returns scores outside the expected 0-100 range, causing the downstream system to interpret 150 as “extremely bullish” and bet the entire portfolio
- The `pool_manager.py` allocates capital to the wrong pool due to a floating-point rounding error
- The NSE data API returns `None` for a stock price, and the algorithm interprets `None` as 0, thinking the stock has crashed 100% and aggressively shorts it

Worst case for TradePilot: The algorithm interprets corrupted data as “market crashing” and places aggressive short positions (bets that stocks will fall) on 20 large-cap stocks. The market is actually rising. By the time the error is detected, losses are Rs 3-5 lakh on a Rs 50 lakh capital base.

Likelihood: HIGH (every algorithmic trading system encounters this eventually)

The solution:

1. **Shadow mode (mandatory first 30 days)** – the live system runs alongside the paper trading system. Both receive the same signals. The live system calculates what it WOULD do but does NOT place orders. A human compares both outputs daily. Only after 30 days of matching results does the live system start placing real orders
2. **Order sanity checks** – before every order reaches the Kite API:
 - Is the stock in our approved universe? (Nifty 500 only, never penny stocks)
 - Is the quantity within limits? (see 2.3)
 - Is the direction consistent with the signal? (BUY signal should not generate a SELL order)
 - Is the price within 5% of last known price? (catch stale data)
 - Has this exact order been placed in the last 60 seconds? (catch duplicates)
3. **Signal validation layer** – scores from `composite_scorer.py` must be within 0-100. Any score outside this range triggers an alert and the signal is discarded
4. **Automated daily reconciliation** – at 3:30 PM, compare TradePilot’s internal position log with Zerodha’s actual positions. Any mismatch triggers an alert

2.2 Flash Crash Protection

Threat: The algorithm enters a feedback loop where it aggressively trades during extreme market volatility, amplifying losses.

Analogy: You are driving a car with cruise control. The road suddenly becomes icy. The cruise control does not know about ice – it keeps accelerating to maintain speed, causing you to spin out. A human driver would have slowed down immediately.

How a flash crash feedback loop works:

1. Market drops 2% in 5 minutes (unusual but happens 2-3 times per year)
2. TradePilot's regime detector switches to "BEAR" mode
3. The algorithm starts shorting stocks (betting they will fall further)
4. Other algos in the market are also selling, pushing prices down more
5. TradePilot sees prices falling further, confirms "BEAR" conviction, adds more shorts
6. The market rebounds suddenly (as flash crashes often do)
7. TradePilot's short positions are now losing money rapidly
8. The algorithm tries to exit, but liquidity has dried up – nobody wants to buy what it is selling
9. Large losses compound in seconds

Worst case for TradePilot: The v5 risk manager has circuit breakers (Tier 1-5), but these are based on the pool_manager's P&L tracking. In a flash crash, prices move so fast that by the time the circuit breaker triggers (checking every few seconds), the damage is already done. On Rs 50 lakh capital with 20 leveraged positions, a 5-minute uncontrolled feedback loop could lose Rs 5-10 lakh.

Likelihood: MEDIUM (India sees 1-2 circuit breaker days per year on NSE, but TradePilot-scale flash events are more frequent)

The solution:

1. **Time-based circuit breaker** – if the Nifty index moves more than 1% in any 5-minute window, immediately:
 - Stop placing new orders
 - Do NOT close existing positions (selling into a crash makes it worse)
 - Wait 15 minutes before resuming
2. **Velocity detector** – monitor order frequency. If TradePilot is placing more than 5 orders per minute, something is wrong. Pause and alert
3. **Regime change cooldown** – when the regime detector switches from BULL to BEAR (or vice versa), impose a 30-minute cooldown before the new regime takes effect. This prevents whiplash trading
4. **Maximum daily order count** – hard limit of 50 orders per day. The current v5 system trades 5-15 times per day. If the system tries to place order #51, it is likely in a feedback loop
5. **Human-in-the-loop for extraordinary conditions** – if VIX jumps above 30 (extreme fear), send a Telegram alert and wait for human confirmation before placing any new trades

2.3 Order Size Validation

Threat: A bug causes an order for Rs 50 lakh instead of Rs 5,000.

Analogy: You tell a waiter “5 plates of biryani” but they hear “500 plates.” By the time you realize the mistake, the kitchen has already started cooking 500 plates and you owe the restaurant.

Current TradePilot limits (from paper trading code):

Parameter	Current Value	Notes
CAPITAL_PER_PORTFOLIO	Rs 5,00,000	Per pool in v5
MAX_POSITION_SIZE	Rs 1,00,000	Per stock
KELLY_CAP	25% of pool	Max position as % of pool capital
MAX_POSITIONS	5 per portfolio	Legacy v4 limit
MAX_POSITIONS_TOTAL	20	v5 risk manager limit

The problem: These limits exist in the paper trading engine but are NOT enforced at the Kite API layer. If a bug bypasses the paper trading logic and calls the Kite API directly, there is no safety net.

Worst case for TradePilot: A bug in position_sizer.py calculates quantity as 10,000 shares of TCS (at Rs 3,800 = Rs 3.8 crore) instead of 10 shares (Rs 38,000). Zerodha’s margin system allows you to place orders up to your available margin, which could be Rs 50 lakh with full collateral. The order executes. You now own Rs 3.8 crore worth of TCS on margin. TCS drops 1%. You lose Rs 3.8 lakh in minutes.

Likelihood: MEDIUM-HIGH (fat finger orders are the most common algo trading failure)

The solution:

1. **Hard order limits in the API wrapper** – create a SafeKiteClient wrapper around pykiteconnect that enforces these limits BEFORE the order reaches Zerodha:

Limit	Value	Action If Exceeded
Max order value	Rs 2,00,000	Reject order, alert on Telegram
Max quantity per order	500 shares	Reject order
Max daily total orders value	Rs 10,00,000	Stop trading for the day
Max single stock exposure	20% of total capital	Reject order
	Rs 1,000	Reject (likely a bug)

Limit	Value	Action If Exceeded
Min order value		

2. **Two-layer validation** – the risk manager checks limits first (existing), then the SafeKiteClient checks independently (new layer). Both must approve
3. **Order confirmation delay** – for any order above Rs 50,000, introduce a 3-second delay before execution. During this delay, re-fetch the current market price and re-validate. If the price has changed more than 1% since the order was calculated, reject it

2.4 Double Ordering

Threat: The same buy signal triggers two identical orders, doubling your exposure.

Analogy: You click “Pay” on an online shopping site and the page freezes. You click again. Two payments go through. You just bought two TVs instead of one.

How this happens in trading systems:

- Network timeout during order placement – the order actually went through but the system did not receive confirmation, so it retries
- Two instances of the trading engine running simultaneously (e.g., cron job starts a new instance before the old one finishes)
- The signal engine generates the same signal twice due to a data refresh race condition
- A restart mid-day causes the system to re-process all signals from scratch

Worst case for TradePilot: The system buys 100 shares of HDFC Bank. Network timeout. Retries. Buys another 100 shares. Now you have 200 shares with double the risk exposure. If HDFC Bank drops 2%, your loss is Rs 24,000 instead of Rs 12,000.

Likelihood: HIGH (network timeouts to Zerodha happen multiple times per day during market hours)

The solution:

1. **Order ID tracking** – before placing any order, generate a unique order ID (e.g., TP-20260414-HDFCBANK-BUY-001). Log this ID. Before placing any new order, check if an order with the same stock + direction was placed in the last 5 minutes
2. **Zerodha order tag** – Kite API supports a tag field in order placement. Use this to tag every order with a unique ID. Query pending orders before placing a new one
3. **Single instance lock** – use a file-based lock (or Redis lock) to ensure only one instance of the trading engine runs at a time. If a second instance starts, it should detect the lock and exit immediately
4. **Post-order verification** – after placing an order, immediately query Zerodha’s order book. Verify the order appears exactly once. If it appears twice, cancel the duplicate

2.5 Market Manipulation Charges

Threat: SEBI (Securities and Exchange Board of India) considers your algorithm's trading pattern as market manipulation.

Analogy: You are at an auction and you keep bidding just to drive the price up, then withdraw your bids at the last moment. This is illegal because it misleads other bidders. An algorithm can accidentally do the same thing.

Actions SEBI considers manipulative:

Practice	What It Looks Like	How TradePilot Could Trigger It
Spoofing	Placing orders you intend to cancel to move the price	Algorithm places limit orders, market moves, algorithm cancels and re-replaces at new price rapidly
Layering	Multiple orders at different prices to create fake demand	Position sizer splits a large order into 5 smaller orders at slightly different prices
Wash trading	Buying and selling the same stock to create fake volume	Algorithm buys a stock, regime changes, algorithm sells the same stock 2 minutes later
Front-running	Trading ahead of a large order you know about	Not applicable to TradePilot (we do not have access to others' orders)
Price manipulation	Large orders that artificially move the stock price	Unlikely at TradePilot's capital size, but possible in illiquid stocks

Worst case for TradePilot: SEBI notices that TradePilot repeatedly buys and sells the same stock within minutes (the INTRADAY pool's normal behavior). They investigate, find no human oversight, and issue a show-cause notice. Even if not guilty, the legal defense costs Rs 5-10 lakh and takes 1-2 years. Meanwhile, Zerodha may freeze the account.

Likelihood: LOW-MEDIUM (more likely as trading volume increases)

The solution:

1. **Minimum hold period** – never sell a stock within 5 minutes of buying it. This alone eliminates most wash trading concerns
2. **Cancel-to-order ratio** – track how many orders are placed vs cancelled each day. If more than 30% of orders are cancelled, reduce trading aggressiveness
3. **Audit trail** – log every order with timestamp, reason (which signal triggered it), and outcome. SEBI will ask for this
4. **Legal opinion** – before going live, get a written legal opinion from a SEBI-registered compliance consultant confirming your algorithm's behavior does not constitute market manipulation
5. **Volume awareness** – never let TradePilot's order volume exceed 1% of a stock's daily traded volume. For small-cap stocks, this limit should be even lower (0.1%)

2.6 Slippage and Execution Risk

Threat: The price at which you expect to buy/sell is different from the price you actually get.

Analogy: You see a pair of shoes priced at Rs 2,000 in a shop window. By the time you walk in and ask for it, the shopkeeper says “Sorry, price just went up to Rs 2,100.” In the stock market, this happens in milliseconds.

Types of slippage:

Type	Cause	Typical Impact
Market order slippage	You say “buy at whatever the current price is” – by the time the order reaches the exchange, the price has moved	0.05-0.5% per trade
Liquidity slippage	The stock does not have enough buyers/sellers at your desired price	0.1-2% in mid/small caps
Latency slippage	Your order takes 200ms to reach the exchange; in 200ms, the price changed	0.01-0.1% per trade
Gap slippage	The stock opens at a different price than yesterday’s close (overnight gap)	1-5% in extreme cases

Why this matters for TradePilot: The current paper trading engine assumes perfect execution – it uses the “last traded price” as the buy/sell price. In reality, with a Rs 1 lakh order: - Buying at the “market price” might cost 0.1-0.3% more than expected - Selling at the “market price” might receive 0.1-0.3% less - Combined slippage of 0.2-0.6% per round trip (buy + sell)

On TradePilot’s Rs 50 lakh capital with 10 trades/day: - Average slippage: 0.3% per round trip - 10 trades x Rs 1 lakh average x 0.3% = Rs 3,000/day in slippage - Monthly: Rs 66,000 – this is pure friction that eats into profits

Worst case for TradePilot: During the April 9 crash (which TradePilot already experienced in paper trading), slippage in live trading would have been 2-5x worse than normal because everyone is selling simultaneously. The v5 paper trading showed a loss of Rs 1,500 on that day – in live trading with slippage, the loss could have been Rs 5,000-10,000.

Likelihood: CERTAIN (slippage is not a question of “if” but “how much”)

The solution:

1. **Limit orders instead of market orders** – always specify the maximum price you are willing to pay (buy) or minimum you will accept (sell). Accept that some orders will not execute rather than pay excessive slippage
 2. **Slippage budget** – add 0.3% to the cost assumption in all P&L calculations. If a trade is only profitable by 0.2% before slippage, skip it
 3. **Smart order routing** – for orders above Rs 50,000, split into 2-3 smaller orders placed 30 seconds apart. This reduces market impact
 4. **Avoid the first and last 15 minutes** – 9:15-9:30 and 3:15-3:30 have the highest slippage due to extreme volume. The current paper engine already avoids entry before 9:30 – maintain this
 5. **Slippage tracking** – log the expected price vs actual execution price for every trade. If average slippage exceeds 0.5%, investigate and tighten limit order spreads
-

CATEGORY 3: Network Security

Every order TradePilot places travels over the internet from your server to Zerodha's servers. That journey has vulnerabilities.

3.1 Man-in-the-Middle (MITM) Attacks

Threat: An attacker positions themselves between TradePilot and Zerodha, intercepting and potentially modifying API calls.

Analogy: You send a letter to your stockbroker through a courier. Someone intercepts the courier, opens the letter, changes "sell 100 shares" to "sell 10,000 shares," seals it back up, and delivers it. Neither you nor the broker knows the letter was tampered with.

How this could work:

1. Attacker compromises the network between your server and Zerodha
2. When TradePilot sends: `POST /orders {"stock": "RELIANCE", "qty": 10, "type": "BUY"}`
3. Attacker intercepts and changes to: `POST /orders {"stock": "PENNYSTOCKXYZ", "qty": 10000, "type": "BUY"}`
4. Zerodha receives the modified request and executes it
5. TradePilot receives a success response (the attacker may even modify the response to show the original order details)

Mitigations already in place: Zerodha's Kite API uses HTTPS (TLS encryption), which means all traffic is encrypted. An attacker cannot read or modify encrypted traffic without breaking the encryption – which is extremely difficult with modern TLS.

But TLS is not bulletproof:

Weakness	How	Likelihood
Compromised Certificate Authority	A trusted CA is hacked and issues a fake certificate for <code>api.kite.trade</code>	Very LOW (has happened historically – DigiNotar 2011)
TLS downgrade attack	Force the connection to use TLS 1.0 (which has known vulnerabilities)	LOW (Zerodha enforces TLS 1.2+)
Server misconfiguration	Your server accepts weak cipher suites	MEDIUM (depends on server setup)

Worst case for TradePilot: An attacker on the same cloud network (if using AWS/GCP) intercepts API calls and modifies order details. This is extremely unlikely but catastrophic if it happens – every order could be manipulated.

Likelihood: LOW

The solution:

1. **TLS certificate pinning** – hardcode the SHA-256 fingerprint of Zerodha’s TLS certificate in your code. If the certificate presented during connection does not match the pinned fingerprint, refuse to connect. Update the pin when Zerodha rotates their certificate (they announce this)
2. **TLS 1.3 enforcement** – configure your HTTP client to only accept TLS 1.3. Reject any connection that tries to negotiate TLS 1.2 or lower
3. **Request signing** – sign every API request with a private key stored in your HSM. Even if an attacker intercepts the request, they cannot modify it without invalidating the signature. (Note: Kite API already uses checksum validation for login, but individual orders are not signed)

3.2 DNS Hijacking

Threat: An attacker redirects `api.kite.trade` to their own server.

Analogy: You look up a restaurant’s address in the phone book. Someone has replaced the phone book entry with a different address – a fake restaurant that looks identical to the real one. You walk in, hand over your credit card, and order food that never arrives.

How DNS hijacking works:

1. Your server asks the DNS resolver: “What is the IP address of `api.kite.trade`?”
2. Normal answer: `203.0.113.50` (Zerodha’s real server)
3. Hijacked answer: `198.51.100.99` (attacker’s server)
4. Your server connects to the attacker’s server, thinking it is Zerodha
5. Attacker receives your API key, access token, and all order details

Worst case for TradePilot: The attacker sets up a perfect replica of the Kite API. TradePilot sends orders to the fake server. The attacker receives the access token and uses it to place their own orders on the real Zerodha API. Meanwhile, TradePilot thinks orders are being executed but nothing is happening on the market.

Likelihood: LOW (requires compromising DNS infrastructure or your server's DNS settings)

The solution:

1. **DNS over HTTPS (DoH)** – configure your server to use encrypted DNS (Cloudflare 1.1.1.1 or Google 8.8.8.8 over HTTPS). This prevents DNS responses from being tampered with
2. **Hardcoded IP fallback** – maintain a hardcoded list of Zerodha's known IP addresses. If DNS resolution returns an unexpected IP, fall back to the hardcoded list and alert
3. **DNSSEC validation** – enable DNSSEC on your DNS resolver. This cryptographically verifies that DNS responses have not been tampered with
4. **Certificate pinning (from 3.1)** – even if DNS is hijacked, the attacker's server cannot present Zerodha's real TLS certificate. Certificate pinning catches this

3.3 SSL/TLS Certificate Pinning (Deep Dive)

What it is: Certificate pinning is like carrying a photograph of your stockbroker. Every time you meet someone claiming to be your broker, you check their face against the photograph. If it does not match, you refuse to do business.

Implementation for TradePilot:

Step 1: Obtain Zerodha's current certificate fingerprint
`openssl s_client -connect api.kite.trade:443 | openssl x509 -fingerprint -sha256`

Step 2: Store this fingerprint in your application config (encrypted)

Step 3: Before every API call, verify the server's certificate matches the stored fingerprint

Step 4: If mismatch: refuse connection, alert on Telegram, log the event

Step 5: Maintain 2–3 backup fingerprints (Zerodha may rotate certificates)

Likelihood of needing this: LOW (but the damage is catastrophic if you do not have it)

The solution: Implement certificate pinning in the SafeKiteClient wrapper. This takes 50 lines of code and eliminates an entire category of attacks.

3.4 VPN Requirements

Threat: Running the trading engine on an unprotected internet connection.

Analogy: You are shouting your stock orders across a crowded room. Everyone can hear what you are buying and selling.

When to use a VPN:

Scenario	VPN Needed?	Why
Cloud server (AWS/GCP)	NO	Already on a private network with direct peering to ISPs
Home computer/server	YES	ISP can log all API traffic; shared neighborhood network
Coffee shop / co-working space	ABSOLUTELY YES	Dozens of strangers on the same WiFi
Mobile device managing the system	YES	Mobile networks are less secure

The solution:

1. **Run on a dedicated cloud server** – this eliminates the need for a VPN entirely
2. **If running locally** – use WireGuard VPN to a cloud endpoint, and route all Zerodha API traffic through the VPN tunnel
3. **Never manage the trading system from public WiFi** – even checking positions on your phone at a cafe is risky if the app sends credentials

3.5 What Happens If Internet Drops Mid-Trade

Threat: Your internet connection fails while you have open positions.

Analogy: You are in the middle of a phone call with your broker saying “sell everything now” – and the phone line goes dead. Your broker does not know you wanted to sell. The market keeps moving.

Scenarios:

Situation	What Happens	Impact
Internet drops, all stop-losses are pre-placed on Zerodha	Stop-losses still work (they live on Zerodha’s servers)	LOW – positions are protected

Situation	What Happens	Impact
Internet drops, stop-losses are managed by TradePilot's code	Stop-losses do NOT trigger – TradePilot cannot send the sell order	HIGH – unlimited downside
Internet drops during order placement	Order may or may not have reached Zerodha	MEDIUM – unknown state
Internet drops and reconnects	TradePilot resumes but has stale data; may make wrong decisions	MEDIUM

Worst case for TradePilot: The current v5 risk manager checks stop-losses locally (in Python code). If the internet drops, these stop-losses are useless. With 20 open positions and no stop-loss protection for 30 minutes, a 2% market move means Rs 1 lakh in unprotected losses.

Likelihood: MEDIUM-HIGH (internet outages during market hours happen 2-3 times per month, even on good connections)

The solution:

1. **Place stop-loss orders on Zerodha, not just in your code** – when entering a position, immediately place a SL order on Zerodha's system. This stop-loss lives on Zerodha's server and executes even if your system is offline
2. **Dual connectivity** – use two internet connections (e.g., fiber + 4G dongle) with automatic failover. If the primary connection drops, switch to backup within 5 seconds
3. **Heartbeat monitoring** – ping Zerodha's API every 30 seconds. If 3 consecutive pings fail, the system is effectively offline. Trigger an alert (SMS, not Telegram – because Telegram needs internet too)
4. **Graceful degradation** – if internet is lost, do NOT try to reconnect and trade immediately. Wait for stable connectivity (30 seconds of uninterrupted connection) before resuming. Stale data after a reconnection is dangerous
5. **Daily reconciliation** – whether or not outages occur, reconcile internal state with Zerodha at least once per hour during market hours

CATEGORY 4: Infrastructure Threats

The server running TradePilot is a single point of failure. If it goes down during market hours, you cannot trade. If it is compromised, an attacker can trade on your behalf.

4.1 Server Compromise (SSH Access)

Threat: An attacker gains SSH access to the server running TradePilot.

Analogy: A thief gets a copy of the key to your office. They can now enter anytime, open your safe (which contains your stockbroker credentials), and do whatever they want.

What an attacker with SSH access can do:

- Read the API secret and access token from wherever they are stored
- Modify the trading algorithm to their benefit (e.g., buy stocks they are selling)
- Exfiltrate all trade history, P&L data, and strategy details
- Install a backdoor for persistent access even after you change the password
- Delete audit logs to cover their tracks

Worst case for TradePilot: The attacker modifies `risk_manager.py` to disable circuit breakers and increases `MAX_POSITION_SIZE` to Rs 50 lakh. The algorithm places massive orders. By the time you check the server, the attacker has restored the original code and deleted the git diff. You have Rs 10 lakh in unexplained losses and no evidence.

Likelihood: LOW-MEDIUM (depends entirely on server hardening)

The solution:

1. **SSH key-only authentication** – disable password login entirely. Use Ed25519 SSH keys (stronger than RSA). The private key stays on your laptop with a passphrase
2. **Fail2ban** – automatically block IP addresses that attempt more than 3 failed SSH logins. This stops brute-force attacks
3. **Non-standard SSH port** – change SSH from port 22 to a random port (e.g., 49152). This eliminates 99% of automated attacks
4. **File integrity monitoring** – install AIDE or Tripwire that hashes every file in the trading system directory. If any file changes unexpectedly, alert immediately
5. **Immutable audit logs** – send all trading logs to a separate logging server (or cloud logging service) in real-time. Even if the attacker deletes logs on the trading server, the copies on the logging server remain
6. **Two-person rule for deployments** – any code change to the trading system requires two people to approve (even if the “second person” is a scheduled review)

4.2 DDoS During Market Hours

Threat: A Distributed Denial of Service attack overwhelms your server with fake traffic, preventing it from communicating with Zerodha.

Analogy: You are in a phone booth trying to call your broker. Someone sends 10,000 people to the same phone booth, all trying to use the phone at the same time. You cannot get through.

Why someone would DDoS a retail trading system:

- A competitor who knows your strategy wants to prevent you from trading
- Extortion (“pay us or we keep attacking”)
- Collateral damage from a broader attack on your cloud provider
- An ex-employee with a grudge

Worst case for TradePilot: The server cannot communicate with Zerodha during a market crash. Your positions have no stop-loss protection (because stop-losses were not pre-placed on Zerodha – see 3.5). By the time you regain connectivity, positions have moved 3% against you. Loss: Rs 1.5 lakh.

Likelihood: LOW (retail trading systems are rarely targeted, but cloud infrastructure outages have the same effect)

The solution:

1. **Cloud provider DDoS protection** – AWS Shield, GCP Cloud Armor, or Cloudflare are included free or cheap with cloud hosting. They absorb volumetric attacks automatically
2. **Pre-placed stop-losses on Zerodha** (critical – this keeps appearing because it is THE most important protection)
3. **Failover server** – maintain a cold standby server on a different cloud provider. If the primary goes down, the standby can be activated within 5 minutes
4. **Minimal attack surface** – the trading server should have NO public-facing ports except SSH (and even that should be behind a VPN or IP whitelist). It talks to Zerodha's API (outbound) and receives Telegram commands (outbound). No inbound connections needed

4.3 Database Corruption

Threat: The file storing position data, trade history, or risk state gets corrupted.

Analogy: Your accountant's ledger book gets water-damaged. Some entries are smudged beyond recognition. You do not know if you owe someone money or they owe you.

Current TradePilot data storage: TradePilot v5 stores all state in JSON files under docs/paper-trades/v5/. This includes pool allocations, position data, risk events, and daily P&L. These are simple text files – no database.

How corruption happens:

Cause	How	Likelihood
Power failure during file write	File is half-written – invalid JSON	MEDIUM
Disk full	Write fails silently, file is truncated	MEDIUM
Concurrent writes	Two processes write to the same file simultaneously	HIGH (if multiple instances)
Accidental deletion	rm *.json instead of rm *.tmp	LOW

Worst case for TradePilot: The position file is corrupted mid-day. TradePilot loses track of its open positions. It does not know what it owns. It may: - Re-buy stocks it already holds (doubling exposure) - Fail to sell stocks at stop-loss because it does not know about them - Report wrong P&L to the founder

Likelihood: MEDIUM-HIGH (JSON file storage with no transaction safety is fragile)

The solution:

1. **Move to a proper database** – replace JSON files with PostgreSQL or SQLite. These databases have built-in transaction safety – if a write fails midway, the database rolls back to the last good state
2. **Write-ahead logging** – before modifying any file, write the intended change to a separate log file first. If the main file gets corrupted, replay the log to rebuild it
3. **Atomic writes** – write to a temporary file first, then rename it to the real filename. Renames are atomic on most filesystems – the file is either the old version or the new version, never a half-written mess
4. **Zerodha as source of truth** – for position data, always treat Zerodha's positions API as authoritative. Reconcile local state against Zerodha every 15 minutes
5. **Hourly backups** – during market hours, back up the position file every hour to a separate directory and to cloud storage

4.4 Power Failure / System Crash

Threat: The server loses power or crashes during market hours.

Analogy: You are in the middle of a chess tournament. Someone trips over the power cord and all the lights go out. When they come back on, you do not remember whose turn it is or where the pieces were.

What happens when the trading server crashes:

1. All in-memory state is lost (current prices, pending signals, risk counters)
2. Any file writes in progress may be corrupted
3. Open positions on Zerodha are still alive – the exchange does not know your server crashed
4. Stop-losses that exist only in TradePilot's code are no longer active
5. When the server restarts, it needs to figure out what state the market is in

Worst case for TradePilot: Server crashes at 10:00 AM with 15 open positions. No pre-placed stop-losses on Zerodha. Market drops 2% between 10:00 and 10:15 (when you notice and restart the server). $15 \text{ positions} \times \text{average Rs } 1 \text{ lakh} \times 2\% = \text{Rs } 30,000$ in unprotected losses. If the crash coincides with a flash crash, this could be Rs 1-2 lakh.

Likelihood: LOW-MEDIUM (modern cloud servers have 99.95%+ uptime, but it does happen)

The solution:

1. **Pre-placed stop-losses on Zerodha** (third time this appears – it is that important)
2. **Fast recovery startup** – the system should be able to restart and reach full operational state within 60 seconds:
 - Read position state from Zerodha (not from local files)
 - Verify all stop-losses are active on Zerodha
 - Resume monitoring

3. **Process supervisor** – use `systemd` or `supervisord` to automatically restart the trading engine if it crashes
4. **UPS for local servers** – if running on a local machine, use an uninterruptible power supply that provides 30 minutes of backup power
5. **Health check alerting** – an external monitoring service (UptimeRobot, or a simple cron on a different server) pings the trading server every minute. If it does not respond, send an SMS alert

4.5 Clock Synchronization

Threat: The server's clock is wrong, causing orders to be placed at the wrong time or with incorrect timestamps.

Analogy: Your watch is 5 minutes slow. You think it is 3:10 PM but it is actually 3:15 PM – the market close deadline. Your “last minute” sell orders arrive after the market has closed and are rejected.

Why clock accuracy matters in trading:

- Market orders placed after 3:30 PM are rejected by the exchange
- Stop-loss timing is critical – a 1-second drift can mean the difference between executing at the target price and missing it
- Audit trails require accurate timestamps – SEBI may question trades with incorrect times
- TradePilot's force-exit at 15:15 (in paper trading) must happen at exactly 15:15, not 15:20

Worst case for TradePilot: The server clock drifts 3 minutes behind. The force-exit routine triggers at 15:18 actual time. By 15:18, the last-minute volatility has pushed prices against several positions. For INTRADAY positions that must close by end of day, this costs Rs 2,000-5,000 per occurrence.

Likelihood: LOW (cloud servers use NTP by default, but NTP failures do happen)

The solution:

1. **NTP synchronization** – ensure the server runs `chrony` or `ntpd` synchronized to multiple NTP servers (`time.google.com`, `time.cloudflare.com`, `ntp.nse.co.in` if available)
 2. **Clock drift monitoring** – alert if the system clock differs from NTP time by more than 1 second
 3. **Use exchange timestamps** – for all trade timing decisions, use the timestamp from Zerodha's API response, not the local clock
 4. **Market hours verification** – before the first trade of the day, compare local time with Zerodha's server time (available in API responses). If they differ by more than 2 seconds, do not trade until synchronized
-

CATEGORY 5: Quantum Computing Threats

This category is about the future. Quantum computers do not currently threaten TradePilot, but the decisions you make today about encryption determine whether your data is safe 5-10 years from now.

5.1 What Is Quantum Computing and Why It Matters

Simple explanation: Regular computers think in “bits” – each bit is either a 0 or a 1, like a light switch that is either off or on. A quantum computer uses “qubits” that can be 0, 1, or both simultaneously (a concept called superposition). This lets quantum computers try millions of solutions at the same time instead of one at a time.

Analogy: Imagine you need to find a specific book in a library with 1 million books.

- A regular computer checks one book at a time. It might take 500,000 checks on average
- A quantum computer creates “ghost copies” of itself that check all 1 million books simultaneously. It finds the book almost instantly

Why this matters for security: Modern encryption works because it would take a regular computer billions of years to crack. A quantum computer could crack it in hours or days.

Current state of quantum computing (April 2026):

Milestone	Status
IBM 1,000+ qubit processor	Achieved (2023)
Google “quantum supremacy”	Achieved (2019, expanded 2024)
Practical quantum computer for cryptanalysis	NOT YET – estimated 2030-2035
Quantum computers available to hackers	NOT YET – currently requires \$10M+ labs
NIST post-quantum encryption standards	FINALIZED (August 2024)

Bottom line: Quantum computers cannot crack your encryption TODAY. But they will be able to within 5-10 years.

5.2 “Harvest Now, Decrypt Later”

Threat: An attacker captures your encrypted data TODAY and stores it. In 5-10 years, when quantum computers are available, they decrypt everything.

Analogy: A thief photographs every page of your diary, which is written in a code they cannot crack today. They put the photos in a vault. In 10 years, they get a code-breaking machine and read every page. Your secrets from 10 years ago are now exposed.

What TradePilot data is at risk:

Data Type	Value to Attacker in 2031-2035
Trading strategy/algorithm	LOW (strategy will have evolved by then)
API keys and secrets	NONE (rotated long before then)
Historical trade data	MEDIUM (reveals patterns, risk tolerance, capital size)
P&L records	MEDIUM (financial intelligence, tax records)
Client data (future)	HIGH (if TradePilot manages others' money)
Communication logs	MEDIUM (business intelligence, partnerships)

Worst case for TradePilot: Not applicable today. BUT if TradePilot grows into a fund management platform (managing other people's money), historical trade records become subject to regulatory retention (7 years for SEBI). Data captured in 2026 and decrypted in 2033 could expose client identities, trade patterns, and financial information – triggering data protection violations.

Likelihood of “harvest now” happening: MEDIUM (nation-state actors and organized crime groups are already doing this with financial data)

Likelihood of “decrypt later” succeeding: HIGH (if data is captured, quantum decryption is a matter of when, not if)

The solution:

1. **Start using quantum-resistant encryption now** for data at rest (stored files, databases). The NIST standards are finalized – there is no reason to wait
2. **Encrypt trade logs and historical data** with AES-256 (quantum-resistant for symmetric encryption) rather than leaving them as plain JSON files
3. **Implement forward secrecy** in all TLS connections – this ensures that even if the encryption key is cracked later, past communications remain protected

5.3 Which Current Algorithms Are Quantum-Vulnerable

Simple explanation: Not all encryption is equally threatened by quantum computers.

Algorithm	Type	Used For	Quantum Threat
RSA-2048	Asymmetric (public/private key)	HTTPS certificates, API authentication	BROKEN by quantum computers
ECC (Elliptic Curve)	Asymmetric	Modern HTTPS, Bitcoin, Kite API TLS	BROKEN by quantum computers
AES-128	Symmetric (same key both sides)	File encryption, database encryption	WEAKENED (effectively halved to 64-bit – not safe)
AES-256	Symmetric		

Algorithm	Type	Used For	Quantum Threat
		File encryption, database encryption	SAFE (reduced to 128-bit equivalent – still strong enough)
SHA-256	Hash function	Password storage, checksums	WEAKENED but still usable
Diffie-Hellman	Key exchange	Setting up encrypted connections	BROKEN by quantum computers

What this means practically:

- The HTTPS connection between TradePilot and Zerodha (which uses RSA or ECC certificates) will eventually be breakable
- Your stored passwords (if hashed with SHA-256) will be harder to crack but not impossible
- Data encrypted with AES-256 will remain safe even against quantum computers
- Data encrypted with AES-128 will NOT be safe

5.4 Quantum-Resistant Algorithms (The Replacements)

In August 2024, NIST (the US National Institute of Standards and Technology) finalized three quantum-resistant encryption standards. These are the replacements for RSA and ECC.

Algorithm	Replaces	Purpose	Status
ML-KEM (CRYSTALS-Kyber)	RSA/ECC key exchange	Securely sharing encryption keys	NIST FIPS 203 – Final Standard
ML-DSA (CRYSTALS-Dilithium)	RSA/ECC digital signatures	Verifying identity and data integrity	NIST FIPS 204 – Final Standard
SLH-DSA (SPHINCS+)	RSA/ECC digital signatures (backup)	Alternative signature scheme using hash functions	NIST FIPS 205 – Final Standard
FN-DSA (FALCON)	RSA/ECC digital signatures	Compact signatures for constrained environments	Expected 2025

How these work (simplified):

- **CRYSTALS-Kyber (ML-KEM):** Based on “lattice” math problems. Think of it like a puzzle involving a multi-dimensional grid where finding the shortest path is astronomically hard – even for quantum computers. Used to agree on an encryption key between two parties
- **CRYSTALS-Dilithium (ML-DSA):** Also lattice-based. Used for digital signatures – proving that a message really came from who it claims to come from, and has not been tampered with

- **SPHINCS+ (SLH-DSA):** Based on hash functions (mathematical one-way functions). A backup option in case lattice-based math is ever broken. Slower but based on completely different mathematics

5.5 What TradePilot Needs To Do About Quantum Threats

Timeline for action:

Timeframe	Action	Priority
Now (2026)	Use AES-256 for all data at rest (trade logs, config files, database)	HIGH
Now (2026)	Ensure all TLS connections use TLS 1.3 (has forward secrecy built in)	HIGH
2026-2027	Migrate API authentication to use hybrid cryptography (traditional + post-quantum)	MEDIUM
2027-2028	When Zerodha/cloud providers support post-quantum TLS, enable it	MEDIUM
2028-2030	Full migration to post-quantum cryptography for all communications	LOW (will be standard by then)

The practical reality: TradePilot does not need to implement CRYSTALS-Kyber today. What it needs to do:

1. **Use AES-256 for everything stored** – this is quantum-safe and available today
2. **Use TLS 1.3** – this has forward secrecy, meaning each session uses a unique key. Even if the long-term key is cracked later, individual sessions remain protected
3. **Watch for Zerodha's quantum-safe upgrades** – when Zerodha updates their API to support post-quantum TLS, adopt it immediately
4. **Do not store secrets in plain text** – today's plain text file is tomorrow's quantum-crackable time bomb

5.6 Quantum-Safe TLS and API Communication

Current state: Zerodha's Kite API uses TLS 1.2/1.3 with ECC or RSA certificates. This is secure today but will be quantum-vulnerable in 5-10 years.

What “quantum-safe TLS” looks like:

Today's TLS handshake:

Client (TradePilot) -> Server (Zerodha): "Let's use ECC to agree on a key"

Both sides compute a shared AES key using ECC math

All further communication encrypted with this AES key

Quantum-safe TLS handshake:

Client -> Server: "Let's use ML-KEM (Kyber) to agree on a key"

Both sides compute a shared AES-256 key using lattice math

All further communication encrypted with AES-256

Hybrid approach (recommended for transition):

Client -> Server: "Let's use BOTH ECC AND ML-KEM"

Shared key = combine(ECC result, ML-KEM result)

This way, even if one algorithm is broken, the other still protects you

When will this be available? - Chrome and Firefox already support hybrid post-quantum TLS (X25519Kyber768 – available since late 2024) - Major cloud providers (AWS, GCP) are rolling out post-quantum TLS in 2025-2026 - Zerodha has not announced post-quantum support yet – but they will adopt it when their TLS provider (likely Cloudflare or AWS) enables it

What TradePilot should do: Configure the HTTP client to prefer hybrid post-quantum cipher suites when available. This is a configuration change, not a code change.

CATEGORY 6: Data Protection

With live trading, TradePilot will handle real financial data – actual profit/loss, actual account balances, actual trading strategies. This is the highest sensitivity data classification.

6.1 Real P&L Data Sensitivity

What changes from paper to live:

Data Type	Paper Trading	Live Trading	Sensitivity
Trade history	Simulated – no real value	Legal financial record	CRITICAL
P&L figures	Hypothetical numbers	Actual taxable income/loss	CRITICAL
Account balance	Fake money	Real bank-linked funds	CRITICAL
Position data	Educational	Market-moving if leaked (front-running risk)	HIGH

Data Type	Paper Trading	Live Trading	Sensitivity
Strategy parameters	Interesting	Commercially valuable trade secret	HIGH
Alert messages	Informational	Real-time trading signals (could be sold)	HIGH

Why real P&L data is dangerous:

- **Tax implications** – manipulated P&L data could lead to incorrect tax filings (fraud charges)
- **Legal evidence** – in any SEBI investigation, your trade logs are evidence. Tampered logs = obstruction of justice
- **Competitive intelligence** – a competitor who sees your P&L knows exactly how well your strategy works and can reverse-engineer the approach
- **Personal risk** – if someone knows you made Rs 50 lakh in profits, you become a target for extortion or social engineering

The solution:

1. **Encrypt all trade data at rest** – use AES-256 encryption for the database and all backup files
2. **Encrypt all trade data in transit** – TLS 1.3 for all API calls (already covered)
3. **Access control** – only the trading engine service account can read/write trade data. No human accounts have direct database access without going through an audit-logged interface
4. **Data classification labels** – tag every data field with its classification (PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED). P&L data is RESTRICTED

6.2 Broker Credentials Storage

Current state (from code review): The Telegram bot token is stored in `telegram_config.json` (a plain text file). This pattern MUST NOT be repeated for Kite API credentials.

Where broker credentials should be stored:

Credential	Storage Method	Access Pattern
API Key	Environment variable (minimum) or Secrets Manager	Read once at startup
API Secret	Hardware Security Module (HSM) or Cloud KMS	Read once per day during token generation
Access Token	Encrypted in-memory only	Generated daily, never written to disk
TOTP Secret	Hardware token (YubiKey)	Never stored digitally on the trading server
Zerodha Password	NEVER stored anywhere	

Credential	Storage Method	Access Pattern
		Entered manually or via secured automation

The ideal setup:

1. API Key and Secret are stored in AWS Secrets Manager (or similar)
2. At 9:00 AM, the trading engine requests the secret from the Secrets Manager (authenticated via IAM role, not a hardcoded key)
3. The engine uses the secret to generate the daily access token
4. The secret is immediately wiped from memory
5. The access token is held in encrypted memory for the day
6. At 6:00 AM next day, the token is discarded

What you MUST NOT do:

- Store credentials in .env files (these get committed to git accidentally)
- Store credentials in Python code
- Store credentials in JSON config files
- Store credentials in Docker environment variables (visible via `docker inspect`)
- Pass credentials as command-line arguments (visible via `ps aux`)

6.3 Trade Logs as Legal Evidence

Why this matters: Under SEBI's algo trading framework, every automated trade must be traceable to:

- The algorithm version that generated the signal
- The data inputs that were used
- The risk checks that were passed
- The exact time of order placement
- The execution price and quantity
- The reason for the trade (which signal, which score)

Current TradePilot logging: The paper trading engine logs to `logs/paper-trade.log` as plain text. This is insufficient for legal purposes.

What live trading logs need:

Requirement	Current State	Required State
Tamper-proof	Plain text file (editable)	Append-only with cryptographic hash chain
Complete	Basic text entries	Structured JSON with all decision inputs
Retained	Overwritten when log rotates	7 years minimum retention
Searchable	grep through files	Indexed database with query interface
Timestamped	Local clock	NTP-synchronized, millisecond precision

Requirement	Current State	Required State
Backed up	Not backed up	3-2-1 backup rule (3 copies, 2 media types, 1 offsite)

The solution:

1. **Structured audit log** – every trading decision is logged as a JSON record containing: timestamp, algorithm_version, signal_scores, risk_check_results, order_details, execution_results
2. **Hash chain** – each log entry includes the SHA-256 hash of the previous entry. This creates a blockchain-like chain where tampering with any entry breaks the chain
3. **Dual storage** – logs written simultaneously to local database AND a cloud logging service (AWS CloudWatch, or a dedicated logging database)
4. **7-year retention** – configure log storage with a 7-year retention policy. Cost is negligible (a year of trading logs is ~1-5 GB)
5. **Monthly log integrity verification** – run a script that verifies the hash chain is unbroken

6.4 Backup Strategy for Trade Data

The 3-2-1 rule: - **3 copies** of all trade data - **2 different media types** (e.g., SSD + cloud storage) - **1 copy offsite** (different geographical location)

Backup schedule for live trading:

Backup Type	Frequency	Where	Retention
Real-time replication	Continuous	Cloud database replica	Current
Hourly snapshot	Every hour during market hours	Cloud storage (S3/GCS)	30 days
Daily full backup	4:00 PM (after market close)	Separate cloud region	1 year
Monthly archive	First day of month	Cold storage (Glacier/Coldline)	7 years
Position snapshot	Every 15 minutes during market hours	Local + cloud	7 days

Recovery time targets:

Scenario	Maximum Acceptable Downtime
Database corruption	15 minutes (restore from hourly snapshot)
Server failure	5 minutes (failover to standby)

Scenario	Maximum Acceptable Downtime
Cloud region outage	30 minutes (restore in different region)
Complete data loss	4 hours (restore from daily backup)

6.5 Right to Audit – SEBI Requirements

What SEBI can demand:

Under the Securities and Exchange Board of India (Intermediaries) Regulations and the upcoming algorithmic trading framework (effective April 2026):

1. **Complete trade records for 7 years** – every order, modification, cancellation, execution
2. **Algorithm logic documentation** – explain in plain English what the algorithm does and why
3. **Risk management documentation** – prove that safeguards exist and are tested
4. **System audit trail** – who accessed the system, when, and what they did
5. **Incident reports** – any time the algorithm malfunctioned, what happened and how it was fixed
6. **Source code** (in extreme cases) – SEBI can request the actual algorithm code

What TradePilot must maintain:

	Requirement	Current Status
☐	7-year trade record retention	NOT IMPLEMENTED (files are not archived)
☐	Algorithm version tracking	PARTIAL (git history exists but not tagged)
☐	Risk management documentation	PARTIAL (risk_manager.py is documented but no formal document)
☐	Access audit trail	NOT IMPLEMENTED
☐	Incident log	NOT IMPLEMENTED
☐	Compliance documentation	NOT IMPLEMENTED

CATEGORY 7: Operational Security

These are the day-to-day operational threats – the things that go wrong during normal use, not because of attackers, but because of design gaps.

7.1 Kill Switch – Emergency Stop

Threat: The algorithm is behaving erratically and you need to stop ALL trading immediately.

Analogy: Every factory has a big red button that stops all machines instantly. In an emergency, you do not want to find individual switches for each machine – you need everything to stop NOW.

Current TradePilot state: The v5 risk manager has a Tier 5 “ALL-STOP” that triggers automatically when monthly portfolio loss exceeds 7%. But there is no manual kill switch that a human can press at any time.

What the kill switch must do (in order):

1. **Cancel all pending orders** on Zerodha (orders placed but not yet executed)
2. **Place market sell orders** for all open positions (or simply cancel the working orders and let stop-losses handle it)
3. **Disable all signal processing** – the algorithm stops generating new signals
4. **Send alert** on Telegram, SMS, and email: “KILL SWITCH ACTIVATED by [who] at [time] – reason: [manual/auto]”
5. **Log the event** with full system state snapshot
6. **Lock out** – prevent any automated trading from resuming until a human explicitly re-enables it with a confirmation code

How to trigger it:

Method	Speed	For When
Telegram command: /killswitch	2-5 seconds	You are on your phone
API endpoint: POST /api/killswitch	<1 second	Automated monitoring detects anomaly
Config file: set KILL_SWITCH=true	10-30 seconds (next check cycle)	SSH access to server
Zerodha dashboard: manually cancel all orders	1-2 minutes	All else fails

The solution:

1. **Implement all four kill switch methods** listed above
2. **Test the kill switch weekly** – every Monday morning before market opens, activate the kill switch and verify it cancels test orders correctly
3. **Dead man’s switch** – if the trading engine does not report a “heartbeat” for 5 minutes, automatically trigger the kill switch. This handles the case where the server crashes and nobody is watching

7.2 Position Limit Enforcement

Threat: The algorithm accumulates positions that exceed safe limits.

Current limits in TradePilot v5:

Limit	Value	Enforced In
Max positions total	20	risk_manager.py
Max same sector	3	risk_manager.py

Limit	Value	Enforced In
Kelly cap	25% of pool per position	risk_manager.py
Max position size	Rs 1,00,000	paper-trade-engine.py

What is missing:

Limit Needed	Why
Max single stock as % of total capital	Prevent 50% of capital in one stock
Max F&O exposure (notional)	F&O positions have leverage – Rs 1L margin can control Rs 10L of stock
Max overnight exposure	Intraday positions should close by 3:15 PM – enforce this
Max correlated positions	5 banking stocks = 1 bet on banking sector, not 5 independent bets

The solution: Add these limits to the SafeKiteClient wrapper as hard blocks that cannot be bypassed by any part of the algorithm.

7.3 Daily Loss Limit (Hard Stop)

Threat: The algorithm loses more money in a single day than acceptable.

Current state: The paper trading engine has DAILY_LOSS_LIMIT = 15000 (Rs 15,000) and the v5 risk manager has portfolio limits (1% daily = Rs 50,000 on Rs 50L capital).

The problem: These limits only prevent NEW trades. They do not force-close existing losing positions. If you have 20 positions and each loses 1%, the total loss is Rs 1 lakh – well beyond the 1% daily limit.

How a proper daily loss limit works:

1. **Soft limit (0.5% of capital = Rs 25,000):** Alert the founder on Telegram. Reduce new position sizes by 50%. Continue trading existing positions
2. **Hard limit (1% of capital = Rs 50,000):** Stop all new trades. Place trailing stop-losses on all existing positions at 1% below current price
3. **Emergency limit (2% of capital = Rs 1,00,000):** Close all positions immediately at market price. Activate kill switch. Do not resume until next trading day

The solution: Implement the three-tier daily loss limit with automatic escalation.

7.4 Broker API Downtime Handling

Threat: Zerodha's API is down or degraded during market hours.

Zerodha Kite API reliability (historical):

Issue	Frequency	Duration
Full API outage	2-3 times per year	15-60 minutes
Degraded performance (slow responses)	5-10 times per year	30 minutes - 2 hours
WebSocket disconnections	Weekly during high volatility	1-5 minutes
Login issues (morning rush)	Monthly	5-30 minutes

What happens to TradePilot during API downtime:

- Cannot place new orders (signals are generated but cannot be acted on)
- Cannot check current prices (operating on stale data)
- Cannot verify stop-losses are active
- Cannot reconcile positions

The solution:

1. **Detect downtime quickly** – monitor API response times. If average response time exceeds 2 seconds (normal is 100-300ms), mark the API as “degraded”
2. **During downtime: DO NOT close positions** – your pre-placed stop-losses on Zerodha's servers still work. The most dangerous thing you can do is panic-close when the API comes back up with stale data
3. **Queue orders** – if a signal is generated during downtime, queue it. When the API recovers, re-evaluate the signal (prices may have changed) before executing
4. **Fallback data source** – use yfinance or NSE direct data feeds as a backup price source. Not for trading, but for monitoring position health during Zerodha downtime
5. **After recovery: full reconciliation** – query all positions, all orders, and all executed trades. Compare with internal state. Fix any discrepancies

7.5 After-Hours Security (Overnight Positions)

Threat: The SWING and POSITIONAL pools hold positions overnight. Between 3:30 PM and 9:15 AM the next day, your algorithm cannot react to overnight news.

Examples of overnight risks:

- Company announces bad quarterly results at 6 PM – stock gaps down 10% at next opening
- US markets crash overnight – Indian markets open 3% lower
- Government announces policy change at 8 AM – specific sectors impacted before market opens
- Geopolitical event (war, sanctions) during non-market hours

Worst case for TradePilot: You hold 5 POSITIONAL stocks overnight, each worth Rs 2 lakh. A global crisis causes markets to gap down 5% at open. Your stop-losses (set at 2% below last close) are useless because the stock opens directly at 5% below. Loss: $5 \times \text{Rs } 2\text{L} \times 5\% = \text{Rs } 50,000$ in the first second of trading.

The solution:

1. **Pre-market scanning** – run the regime detector at 8:45 AM (before market opens) using global market data (US markets, Asian markets, commodity prices). If a crash is detected, reduce all stop-losses to 1% and prepare to exit at market open
2. **Gap-down protection** – if a stock opens more than 3% below yesterday's close, immediately sell at market price. Do not wait for the stop-loss
3. **Overnight position limits** – limit overnight exposure to 60% of capital. Keep 40% in cash for morning opportunities and protection
4. **News monitoring** – subscribe to Telegram channels that push breaking financial news. Integrate with TradePilot's alert system to flag overnight risks

7.6 Insider Threat

Threat: Someone with access to the codebase manipulates the algorithm for personal gain.

Analogy: A bank employee who knows the vault combination and the security camera blind spots. They do not need to “hack” anything – they already have legitimate access.

Who has access to TradePilot currently:

Person	Access Level	Risk
Founder (you)	Full code + server + Zerodha account	LOW (it is your money)
Co-founder/partner	Code review access	MEDIUM (if they have server access)
Future developer hire	Code access, possibly server access	HIGH
DevPilot system	Automated access to project files	LOW (no Zerodha credentials)

How an insider could exploit access:

1. Modify `signal_engine.py` to always generate BUY signals for a specific stock they hold personally
2. Add a backdoor that forwards all access tokens to an external server
3. Modify `risk_manager.py` to disable circuit breakers
4. Change log files to hide evidence of manipulation

The solution:

1. **Code review for ALL changes** – every pull request must be reviewed by at least one other person before merging to the production branch

2. **Separate access levels** – the person who writes the algorithm should NOT have access to the production server. Deployment should be automated (CI/CD)
 3. **Audit logging that developers cannot modify** – send all trading logs to a service that developers do not have admin access to
 4. **Git signed commits** – require GPG-signed commits so every code change is cryptographically linked to a specific person
 5. **Regular code audits** – quarterly review of the entire codebase for unauthorized changes
-

CATEGORY 8: Social Engineering and Phishing

These attacks target people, not systems. The most sophisticated security can be bypassed by tricking a human.

8.1 Fake Kite Login Pages

Threat: An attacker creates a website that looks exactly like Zerodha's login page. You enter your credentials, and they capture them.

Analogy: Someone builds an exact replica of your bank's entrance. You walk in, swipe your card, enter your PIN, and wonder why the door does not open. Meanwhile, they have copied your card and PIN.

How this could target TradePilot:

1. Attacker sends an email: "Zerodha API key expiring, click here to renew"
2. The link goes to `kite-zerodha.com` (fake) instead of `kite.trade` (real)
3. You enter your API key, secret, and TOTP code
4. Attacker now has full access to generate access tokens

Or more dangerously:

1. Attacker compromises the automated login flow in TradePilot v6
2. Instead of redirecting to `kite.zerodha.com`, the system redirects to a fake page
3. The daily token generation process sends credentials to the attacker
4. The attacker receives fresh access tokens every morning automatically

Likelihood: MEDIUM (phishing is the #1 attack vector globally)

The solution:

1. **Hardcode the Zerodha login URL** in the application – never construct it dynamically from user input or configuration
2. **Verify the URL before any credential submission** – compare against a whitelist: `kite.zerodha.com`, `kite.trade`, `api.kite.trade`
3. **Bookmark the real Zerodha pages** – never click links in emails to reach Zerodha
4. **Use a password manager** (1Password, Bitwarden) – password managers only auto-fill credentials on the REAL domain. If you are on a fake page, the password manager will not offer to fill in your password, which is your warning

5. **Enable Zerodha's 2FA notification** – Zerodha sends an email when your account is logged into. Monitor for unexpected logins

8.2 Phishing for TOTP Codes

Threat: An attacker tricks you into sharing your TOTP (Time-based One-Time Password) code.

Analogy: The security guard at your bank asks for your 6-digit code to let you in. But this person is not actually the security guard – they are wearing a fake uniform. You give them the code, and they rush inside before you.

How TOTP phishing works:

1. Attacker calls pretending to be Zerodha support: “We detected suspicious activity on your account. Please share your TOTP code to verify your identity”
2. You share the 6-digit code from Google Authenticator
3. Attacker enters the code on the real Zerodha login page within the 30-second window
4. They are now logged in as you

For TradePilot v6 specifically: If the automated login process is compromised, the attacker could intercept the TOTP code as it is generated and use it to create their own session.

Likelihood: LOW-MEDIUM (requires social engineering and precise timing)

The solution:

1. **Never share TOTP codes** with anyone, for any reason. Zerodha will NEVER ask for your TOTP code via phone or email
2. **Use a hardware security key (YubiKey)** instead of Google Authenticator. Hardware keys cannot be phished – they verify the website domain before responding
3. **For automated login:** Generate the TOTP code on the server itself using the TOTP secret stored in the HSM. The code never leaves the server
4. **Rate limit TOTP attempts** – Zerodha already does this (3 failed attempts = account locked), but monitor for lockout events as they may indicate someone trying your codes

8.3 SIM Swap Attacks for 2FA

Threat: An attacker convinces your mobile carrier to transfer your phone number to their SIM card. They now receive your SMS OTPs.

Analogy: You move to a new house and set up mail forwarding. Someone goes to the post office, pretends to be you, and forwards YOUR mail to THEIR address. They now receive your bank statements, credit card bills, and OTP letters.

How it works:

1. Attacker gathers your personal information (name, address, date of birth, Aadhaar number – often available on social media or dark web)

2. They visit a mobile store (or call customer care) claiming to be you: “I lost my phone, I need a replacement SIM”
3. The carrier deactivates your SIM and activates a new one for the attacker
4. All SMS messages (including OTPs from Zerodha, bank, and email services) now go to the attacker
5. They reset your Zerodha password using SMS OTP and gain full account access

Worst case for TradePilot: Attacker gains access to Zerodha account via SIM swap. They can: change the linked bank account (takes a few days), place rogue orders, and view all financial data. Since the Zerodha website (not just the API) provides fund transfer capabilities, the financial risk extends beyond just trading.

Likelihood: LOW-MEDIUM (SIM swap attacks in India require Aadhaar verification at carrier stores, making them harder but not impossible)

The solution:

1. **Use app-based 2FA (TOTP)** instead of SMS – already in use for Zerodha login
2. **SIM lock** – contact your mobile carrier and request a “SIM lock” or “port freeze.” This prevents number porting without physical presence and extra verification
3. **Separate phone number** – use a dedicated phone number (not your primary one) for financial accounts. Keep this number private
4. **Monitor SIM status** – if your phone suddenly shows “No Service” for more than 5 minutes, immediately contact your carrier and check if a SIM swap was initiated

8.4 Impersonation of Zerodha Support

Threat: Someone contacts you pretending to be from Zerodha, requesting account access for “verification” or “technical issues.”

What Zerodha support will NEVER ask for:

Zerodha Will NEVER Do This	
☐	Ask for your password over phone/email/chat
☐	Ask for your TOTP code
☐	Ask for your API secret
☐	Send a link to “verify your account” via email/SMS
☐	Ask you to install remote desktop software (TeamViewer, AnyDesk)
☐	Ask for your bank OTP
☐	Call you about “suspicious activity” and ask for credentials

The solution:

1. **Only contact Zerodha through official channels** – support.zerodha.com, support@zerodha.com, or the in-app support chat
2. **If someone claims to be from Zerodha and asks for credentials** – hang up immediately and report to support@zerodha.com

3. **Verify caller identity** – ask them to confirm details only Zerodha would know (your registered email, your client ID starting with specific letters). If they cannot, they are fake
-

CATEGORY 9: Supply Chain Attacks

A supply chain attack compromises a tool or library that TradePilot depends on. You do not get hacked directly – your tools get hacked, and you inherit the compromise.

9.1 Compromised Python Packages

Threat: A Python package that TradePilot uses (yfinance, pykiteconnect, scikit-learn) is compromised, and malicious code is injected.

Analogy: You buy ingredients from a trusted grocery store. One day, someone tampers with the salt on the store shelf, replacing it with a look-alike that contains poison. You use it in cooking without checking because you trust the store.

TradePilot's current dependencies (from requirements.txt):

Package	Purpose	Risk If Compromised
yfinance	Stock data download	Attacker feeds fake prices – algorithm makes wrong decisions
flask	Web server	Attacker gains remote code execution on the server
xgboost	ML model training	Attacker modifies model weights to predict incorrectly
lightgbm	ML model training	Same as xgboost
scikit-learn	ML preprocessing	Data pipeline corrupted – wrong features fed to model
pandas	Data manipulation	Data corruption, wrong calculations
numpy	Numerical computation	Calculation errors leading to wrong position sizing
pykiteconnect (not yet in requirements)	Zerodha API client	CRITICAL – attacker can intercept all API calls, steal tokens

Real-world supply chain attacks:

Incident	Year	Impact
event-stream (npm)	2018	Bitcoin wallet stealing code injected into a popular package
ua-parser-js (npm)	2021	

Incident	Year	Impact
PyPI malicious packages	2023-2026	Crypto miner injected, 8M weekly downloads affected Hundreds of packages with names similar to popular ones (typosquatting)
Codecov breach	2021	CI/CD tool compromised, secrets from 29,000 repos exposed

Worst case for TradePilot: pykiteconnect is compromised. The malicious version silently copies your API key, secret, and access token to an external server whenever `kite.login()` is called. The attacker uses these credentials to trade on your account. You do not notice because the library still works normally – it just has an extra “phone home” call hidden in the code.

Likelihood: LOW-MEDIUM (smaller packages like pykiteconnect have fewer maintainers and less security review than numpy or pandas)

The solution:

1. **Pin exact versions** – in `requirements.txt`, use `pykiteconnect==5.0.1` (exact version) instead of `pykiteconnect>=5.0.0` (any version above). This prevents automatic installation of a compromised newer version
2. **Hash verification** – use `pip install --require-hashes` with a `requirements.txt` that includes SHA-256 hashes for every package. If the package content changes (even by one byte), installation fails
3. **Private mirror** – host your own PyPI mirror with only the packages you need, pre-verified. This prevents typosquatting and supply chain injections
4. **Dependency audit** – run `pip-audit` weekly to check all installed packages against the known vulnerability database (PyPI Advisory Database)
5. **Virtual environment isolation** – never install trading packages in the system Python. Use a dedicated virtual environment with only the required packages

9.2 Malicious Data Injection

Threat: An attacker feeds fake stock price data to TradePilot’s algorithm, causing it to make profitable trades for the attacker.

Analogy: You are an art dealer who decides what to buy based on auction results. Someone creates a fake auction website showing that a particular artist’s paintings are selling for Rs 50 lakh each. You buy one for Rs 40 lakh, thinking it is a bargain. The paintings are actually worth Rs 5 lakh.

How this could target TradePilot:

1. **Compromised data source** – if yfinance’s data source is hacked, or if a man-in-the-middle attack modifies data in transit
2. **Cache poisoning** – TradePilot caches data in `prototype/data/cache/`. If an attacker can write to this directory, they can inject fake data

3. **API response manipulation** – NSE data API responses could be intercepted and modified

The attack scenario:

1. Attacker injects data showing that Stock XYZ has massive institutional buying, strong momentum, and is breaking out above resistance
2. TradePilot's algorithm sees a high composite score and generates a BUY signal
3. Attacker has already placed sell orders for Stock XYZ at inflated prices
4. TradePilot buys Stock XYZ at the inflated price
5. With no real demand, the stock price drops back. TradePilot takes a loss
6. The attacker profits from the sale

Likelihood: LOW (requires either compromising data providers or physical access to the server)

The solution:

1. **Multi-source data validation** – never rely on a single data source. Cross-check prices from yfinance vs NSE direct feed vs Zerodha quotes. If any source differs by more than 1%, flag the data as suspicious
2. **Price sanity checks** – before acting on any price data, verify:
 - Is the price within 20% of the 20-day moving average? (stocks rarely move more than this)
 - Is the volume believable? (100x normal volume on a small-cap is suspicious)
 - Does the price move match the broader market direction? (one stock up 10% while Nifty is down 2% warrants investigation)
3. **File integrity monitoring** – hash the data cache directory and alert if files are modified outside of the data engine's process
4. **Read-only data pipeline** – the data engine writes to the cache; the algorithm reads from it. No other process should have write access

9.3 PyPI Package Hijacking (Typosquatting)

Threat: An attacker publishes a package named pykiteconnect (one “n”) or yfinancee (extra “e”) that contains malicious code. A developer accidentally installs the wrong one.

The solution:

1. **Copy-paste, never type** – always copy package names from official documentation
 2. **Verify package metadata** – before installing a new package, check its PyPI page for: author name, download count, repository link, last updated date. A package with 5 downloads and no repository link is suspicious
 3. **Automated scanning** – use safety check or pip-audit in your CI/CD pipeline
-

CATEGORY 10: Regulatory and Legal

SEBI is actively tightening rules around algorithmic trading. Non-compliance can result in account freezes, penalties, and criminal charges.

10.1 SEBI Algo Trading Rules (2026)

Background: On March 30, 2025, SEBI released the framework for “Algos by Retail Investors” (circular SEBI/HO/MRD/MRD-PoD-2/P/CIR/2025/47). This framework became effective in phases through 2025-2026.

Key requirements that affect TradePilot:

Requirement	What It Means	Impact on TradePilot
Algo registration	All algorithmic trading systems must be registered with the stock exchange through the broker	TradePilot must be registered as an “algo” with Zerodha, who submits it to NSE/BSE
Unique algo ID	Every registered algo gets a unique identifier. All orders must carry this ID	Every order TradePilot places must include the algo registration number
Broker responsibility	The broker (Zerodha) is responsible for ensuring the algo works correctly	Zerodha may require you to pass a compliance review before allowing live algo trading
Order-level tagging	Every automated order must be tagged as “algo” (not manual)	The Kite API order placement must include an <code>algo_tag</code> parameter
Audit trail	Complete record of all algo decisions must be maintained	Already covered in Category 6.3
Kill switch	Algos must have an emergency stop mechanism	Already covered in Category 7.1
Risk controls	Price range checks, quantity limits, order-to-trade ratio limits	Already partially implemented in v5 <code>risk_manager</code>

How to register TradePilot as an algo:

1. Apply to Zerodha’s algo trading program (Console > Kite Connect > Algo Registration)
2. Submit: algorithm description, risk controls, testing results, compliance documentation
3. Zerodha reviews and submits to NSE/BSE for approval
4. Receive a unique algo ID
5. Include this ID in all API order calls

Consequence of NOT registering: Zerodha detects unregistered automated trading (patterns like rapid order placement, consistent timing, lack of human interaction) and may: - Freeze your account - Reverse trades - Report to SEBI - Terminate your API access

10.2 Audit Trail Requirements

What must be logged (per SEBI circular):

	Record	Retention
☐	Every order placed (with timestamp, price, quantity, algo ID)	7 years
☐	Every order modification	7 years
☐	Every order cancellation (with reason)	7 years
☐	Every order execution (with fill price and quantity)	7 years
☐	Algorithm decision inputs (what data triggered the order)	5 years
☐	Risk check results (which checks passed/failed)	5 years
☐	System uptime/downtime records	3 years
☐	Algorithm version history (code changes)	7 years
☐	Kill switch activation events	7 years

10.3 Tax Implications of Automated Trading

Key tax considerations:

Aspect	Rule	TradePilot Impact
STCG (Short-term capital gains)	20% tax on profits from stocks held < 12 months	ALL intraday and swing trades fall here
Intraday profits	Treated as speculative business income, taxed at your slab rate	The INTRADAY pool's profits are taxed as business income, not capital gains
F&O profits		

Aspect	Rule	TradePilot Impact
	Non-speculative business income, taxed at slab rate	If v6 includes F&O (v5.2 already experiments), all F&O profits are business income
Frequent trading	If you trade daily, tax authorities may classify you as a “trader” not “investor”	Changes the tax treatment of ALL your stock holdings (not just TradePilot’s)
Turnover calculation	For F&O, turnover = sum of absolute profits and losses per trade	High turnover may require a tax audit if turnover exceeds Rs 10 crore
Advance tax	If expected tax liability > Rs 10,000, pay advance tax quarterly	TradePilot must track cumulative P&L for advance tax estimation

What TradePilot must build:

1. **Real-time tax tracker** – running STCG, business income, and turnover calculations
2. **Monthly P&L statement** – exportable in format compatible with CA software
3. **Trade-wise tax report** – for every trade: buy date, sell date, holding period, profit/loss, tax classification

10.4 Liability If Algorithm Causes Market Disruption

Scenario: TradePilot’s algorithm malfunctions and places 1,000 orders in 10 seconds on illiquid stocks, causing a brief price spike of 15%. Other market participants suffer losses.

Legal consequences:

Authority	Action	Consequence
SEBI	Investigation for market manipulation	Fine up to Rs 25 crore or 3x profit (whichever is higher)
NSE/BSE	Suspension of trading membership	API access revoked, cannot trade on exchange
Zerodha		

Authority	Action	Consequence
	Freeze account, reverse trades	All funds locked pending investigation
Other traders	Civil lawsuit for losses caused	Personal liability if trading as individual
Criminal court	If manipulation is proven intentional	Up to 10 years imprisonment under SEBI Act

Critical protection: Even though TradePilot is a personal trading system, the moment it goes live with real money through a registered broker, you are subject to ALL market regulation as any other market participant. “My algorithm had a bug” is NOT a defense against market manipulation charges – it shows negligence in testing.

The solution:

1. **Extensive testing** – run the live system in shadow mode for at least 30 days before placing real orders
2. **Circuit breakers** – all the circuit breakers in Category 2 are not just good practice, they are regulatory requirements
3. **Professional indemnity insurance** – explore insurance that covers algorithmic trading errors (this is a niche but growing market)
4. **Legal entity** – consider running TradePilot through a company (LLP or Pvt Ltd) rather than as an individual. This provides limited liability protection
5. **Compliance consultant** – engage a SEBI-registered compliance officer to review the system before going live

10.5 Insurance Requirements

Types of insurance to consider:

Insurance	What It Covers	Estimated Cost
Professional Indemnity	Errors and omissions in the algorithm	Rs 15-30K/year for Rs 50L coverage
Cyber Insurance	Data breaches, system compromises	Rs 10-25K/year for Rs 25L coverage
Business Interruption	Loss of income during system downtime	Rs 5-15K/year
Directors & Officers (if company)	Personal liability protection	Rs 20-50K/year

Is insurance mandatory? Not currently for individual algo traders. But if TradePilot evolves into a fund management platform (managing other people’s money), SEBI requires registered investment advisors to maintain professional indemnity insurance.

Quantum-Grade Security Architecture

This section describes what the ideal secure trading system looks like – the target architecture for TradePilot v6.

Architecture Overview

LAYER 1: PERIMETER DEFENSE

- Cloud firewall (only outbound to Zerodha + data providers)
- DDoS protection (cloud-native)
- IP whitelisting on SSH
- No public-facing ports

LAYER 2: IDENTITY & ACCESS

- SSH key-only with Ed25519 keys
- Hardware security module (HSM) for API secrets
- TOTP generated on-server (never transmitted)
- Service accounts with minimal permissions

LAYER 3: COMMUNICATION SECURITY

- TLS 1.3 with certificate pinning
- Hybrid post-quantum cipher suites (when available)
- DNS over HTTPS
- Request signing for all API calls

LAYER 4: APPLICATION SECURITY

- SafeKiteClient wrapper with hard order limits
- Signal validation layer
- Duplicate order detection
- 3-tier daily loss limits with automatic escalation
- Kill switch (Telegram + API + config + manual)
- Shadow mode for first 30 days

LAYER 5: DATA PROTECTION

- AES-256 encryption at rest (quantum-safe)
- Encrypted database (PostgreSQL with pgcrypto)
- Hash-chained audit logs
- 7-year retention with 3-2-1 backups
- Memory encryption for in-flight credentials

LAYER 6: MONITORING & ALERTING

- Real-time position reconciliation (every 15 min)
- API response time monitoring
- Anomaly detection on order patterns
- Multi-channel alerting (Telegram + SMS + email)
- Dead man's switch (auto kill if heartbeat fails)

LAYER 7: RECOVERY & RESILIENCE

- Pre-placed stop-losses on Zerodha (server-side)
- Dual internet connectivity with auto-failover
- Cold standby server on different cloud provider
- 60-second recovery startup

Hourly position state backups

LAYER 8: COMPLIANCE & AUDIT

SEBI algo registration

Structured audit trail (JSON + hash chain)

Algorithm version tracking (git tags)

Monthly compliance review

Tax tracking and quarterly advance tax

Priority Implementation Roadmap

Priority	Item	Effort	Impact
P0 (Before Live)	Pre-placed stop-losses on Zerodha	2 days	Prevents catastrophic overnight/outage losses
P0 (Before Live)	SafeKiteClient with hard order limits	3 days	Prevents fat-finger and rogue order losses
P0 (Before Live)	Kill switch (all 4 methods)	2 days	Emergency stop capability
P0 (Before Live)	Shadow mode infrastructure	3 days	Validates live vs paper alignment
P0 (Before Live)	SEBI algo registration via Zerodha	2-4 weeks	Legal compliance (cannot trade without it)
P1 (First Month)	Encrypted credential storage (HSM/KMS)	3 days	Protects broker credentials
P1 (First Month)	Structured audit logging with hash chain	5 days	Legal compliance + tamper detection
P1 (First Month)	3-tier daily loss limit	2 days	Automated loss containment
P1 (First Month)	Duplicate order detection	1 day	Prevents double ordering
P1 (First Month)	TLS certificate pinning	1 day	Man-in-the-middle protection
P2 (First Quarter)	PostgreSQL database migration (from JSON files)	1 week	Data integrity and recovery
P2 (First Quarter)		1 day	

Priority	Item	Effort	Impact
	Dual internet connectivity		Network resilience
P2 (First Quarter)	Multi-source data validation	3 days	Fake data protection
P2 (First Quarter)	Tax tracking module	1 week	Tax compliance
P3 (First Year)	AES-256 encryption at rest for all data	3 days	Data protection + quantum resistance
P3 (First Year)	Dependency hash pinning and private mirror	2 days	Supply chain protection
P3 (First Year)	Hybrid post-quantum TLS (when available)	1 day	Future quantum protection
P3 (First Year)	Professional indemnity insurance	1 day	Liability protection

Summary: Threat Severity Matrix

Threat	Category	Likelihood	Impact	Priority
Rogue algorithm / wrong orders	Financial	HIGH	CRITICAL	P0
No pre-placed stop-losses on Zerodha	Operational	CERTAIN	CRITICAL	P0
Fat-finger order (wrong size)	Financial	MEDIUM-HIGH	CRITICAL	P0
No SEBI algo registration	Regulatory	CERTAIN	CRITICAL	P0
No kill switch	Operational	HIGH	CRITICAL	P0
API secret stored in plain text	Kite API	MEDIUM-HIGH	CRITICAL	P0
Internet drops with no server-side SL	Network	MEDIUM-HIGH	HIGH	P0
Double ordering on retry	Financial	HIGH	HIGH	P1

Threat	Category	Likelihood	Impact	Priority
Slippage eating profits	Financial	CERTAIN	MEDIUM	P1
Flash crash feedback loop	Financial	MEDIUM	HIGH	P1
Rate limit causing missed exits	Kite API	HIGH	HIGH	P1
Database corruption (JSON files)	Infrastructure	MEDIUM-HIGH	HIGH	P2
Broker API downtime	Operational	MEDIUM	MEDIUM	P2
Session hijacking	Kite API	LOW-MEDIUM	HIGH	P2
Server compromise (SSH)	Infrastructure	LOW-MEDIUM	CRITICAL	P2
Compromised Python packages	Supply Chain	LOW-MEDIUM	CRITICAL	P2
Tax non-compliance	Regulatory	MEDIUM	HIGH	P2
Market manipulation charges	Regulatory	LOW-MEDIUM	CRITICAL	P2
Man-in-the-middle attack	Network	LOW	CRITICAL	P3
DNS hijacking	Network	LOW	HIGH	P3
DDoS during market hours	Infrastructure	LOW	HIGH	P3
Power failure / system crash	Infrastructure	LOW-MEDIUM	MEDIUM	P3
Clock synchronization	Infrastructure	LOW	LOW	P3
Fake Kite login pages (phishing)	Social Engineering	MEDIUM	HIGH	P3
SIM swap attack	Social Engineering	LOW-MEDIUM	HIGH	P3
Harvest now, decrypt later (quantum)	Quantum	MEDIUM	MEDIUM (future)	P3
Quantum TLS vulnerability	Quantum	LOW (today)	HIGH (future)	P3
Malicious data injection	Supply Chain	LOW	HIGH	P3

Threat	Category	Likelihood	Impact	Priority
Insider threat	Operational	LOW	CRITICAL	P3

Final Recommendation

Do not go live without the P0 items. They represent non-negotiable safety requirements. The P0 list is deliberately short (5 items, ~2 weeks of work) so that it does not delay the live launch unnecessarily.

The most important single item across this entire report is: **place stop-loss orders on Zerodha's servers, not just in your Python code.** This one change protects against internet outages, server crashes, power failures, DDoS attacks, and API downtime – all simultaneously. It is the highest-leverage security investment you can make.

The quantum computing section is included for completeness and because using AES-256 today (instead of AES-128 or no encryption) costs nothing extra and provides protection that lasts decades. It is not urgent, but it is free insurance.

This document should be reviewed quarterly and updated as TradePilot v6 evolves from planning to live trading. Each review should verify that P0 items remain implemented and P1/P2 items are progressing.